

MAPDS Software Structure

Framework that the pseudocode and final code will follow

Purpose: The MAPDS software will be split into three connected areas: the cockpit interface, the launcher controller, and the drone-attached auxiliary communication module. The launcher controller will be the center of the system. It will run the cockpit interface locally, output it to a monitor through HDMI or DisplayPort, control deployment, read launcher sensors, monitor launcher power, and receive LoRa telemetry from the deployed drone module.

1. System Structure

The software will use separate modules with clear jobs. Each module will exchange only the information it needs. This keeps the code easy to change and supports the MOSA requirement for defined module boundaries and replaceable parts. The launcher and drone-attached module will both use their own power, and battery status will be part of the software data.

Cockpit Interface	Launcher / Holding Mechanism	Drone-Attached Module
Runs locally on the launcher computer and is displayed on a cockpit monitor through HDMI or DisplayPort. Displays system state, faults, LoRa status, GPS position, launcher battery, drone-module battery, and available telemetry. Provides Arm and Deploy controls.	Runs the main control software. Hosts the website, reads launcher sensors, monitors its independent battery, performs safety checks, controls the release hardware, runs the state machine, and receives LoRa data from the drone.	Runs on the removable auxiliary communication module attached to the UAV. Reads GPS, IMU, and its own battery/module status, builds a telemetry message, and sends it to the launcher through LoRa.

2. Launcher Software

The launcher will contain the main program and the supporting modules below. The main program will not directly handle every sensor or device. It will call the correct module and use the result. The launcher will also monitor its own battery because the retention system will not depend on aircraft power.

Module	Job
launcher_main	Starts the system and coordinates all launcher modules.
state_manager	Keeps the system in Standby, Armed, Deploying, Deployed, or Fault.
safety_checks	Checks drone presence, launcher power, module status, sensor health, and other required conditions before deployment.
release_control	Controls the release hardware and confirms whether release occurred.
launcher_sensors	Reads the launcher-side presence and release sensors.
power_monitor	Reads the independent launcher/retention-system battery and reports its status to the rest of the software.
lora_receiver	Receives GPS, battery, and other telemetry from the deployed drone module and tracks whether the link is active.
interface_host	Runs the local cockpit interface and provides it with current launcher and drone information.

3. Drone Module Software

The drone module will remain independent from the UAV internal electronics. Its program will be small and focused. It will collect GPS, IMU, battery, and module-status data, check that the data is valid, build one standard telemetry message, and transmit that message to the launcher. The communication link will be designed around the project target of a 2-mile radius in optimal conditions.

- drone_main: starts the module and controls the update cycle.
- drone_sensors: reads GPS, IMU, battery status, and any later approved sensors.
- telemetry_format: keeps the message fields in one fixed order and format.
- lora_transmitter: sends GPS, battery, and other telemetry to the launcher at a fixed update rate and supports the 2J-mile communication target.

4. Cockpit Interface

The cockpit interface will run locally on the launcher computer and will be displayed on a monitor through HDMI or DisplayPort. The interface can still be built as a webpage, but it will not require a network connection for the prototype.

The operator will use it to view system information and send Arm or Deploy requests. The launcher will decide whether each request is allowed.

- Display the current state in text: Standby, Armed, Deploying, Deployed, or Fault.
- Display launcher health, drone presence, LoRa connection status, GPS position, launcher battery, drone-module battery, and available drone telemetry.
- Display faults in clear English. Color can help, but the meaning will not depend on color alone.
- Provide Arm and Deploy controls. Deployment remains blocked when a required check fails.

5. Information Flow

Drone Sensors -> Drone Module -> LoRa -> Launcher -> Cockpit Interface -> HDMI/DisplayPort -> Cockpit Monitor
Operator Command -> Cockpit Interface -> Launcher -> Safety Checks -> Release Hardware

These two paths stay separate. Telemetry moves toward the cockpit display. Operator commands move from the interface into the launcher. The launcher is the only part that makes the final deployment decision. If the LoRa link is lost, the interface will show communication loss instead of using old telemetry as current data.

6. State and Fault Logic

The launcher code will follow the current finite-state-machine concept. The normal sequence is Standby -> Armed -> Deploying -> Deployed. Before deployment, the launcher will check the required sensor, module, and power conditions. If a required check fails, the system enters a fault condition or returns to a safe state. A missing sensor reading, communication problem, or failed health check will not cause a release command.

7. What the Pseudocode Will Represent

The pseudocode will directly represent this structure. We will first write the launcher framework because it connects every part of the system. Then we will write the launcher state logic, safety checks, power monitoring, release control, LoRa handling, cockpit interface behavior, and drone telemetry cycle. Each pseudocode section will match one software module listed in this document so it can later be converted into real code without changing the overall structure.